Intune Custom Compliance & Configuration Items (Scripted) — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 4

The problem . . . 4

The solution . . . 4

How it's delivered . . . 4

The outcome . . . 4

Solution Overview — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 5

What this solution delivers . . . 5

Named objects (create once, reuse everywhere) . . . 5

Commercial vs GCC High — what's different . . . 6

Who this is for . . . 8

Requirements — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 8

R1. GCC High tenant (US sovereign cloud), not Commercial or GCC-Moderate . . . 9

R2. US Government Intune licensing (G3/G5 Gov) . . . 9

R3. Roles: Intune Administrator (+ default scope tag) in the GCC High tenant . . . 9

R4. Microsoft Graph PowerShell SDK connecting to US Gov . . . 9

R5. Graph permission scopes for discovery, validation, reporting, remediation . . . 10

R6. A tested Windows discovery script + JSON rules file . . . 10

R7. Sovereign network endpoints allowed through filtering (no TLS break) . . . 10

R8. Windows devices enrolled in the GCC High tenant, in scope . . . 11

Global Requirements — Intune Solutions (category-level) 11

G1. Microsoft Entra ID tenant . . . 11

G2. Microsoft Intune subscription . . . 11

G3. Administrative roles . . . 12

G4. Device enrollment in Intune . . . 12

G5. Pilot security group . . . 12

G6. Supported Windows baseline . . . 12

G7. Network access to Microsoft cloud endpoints . . . 12

Implementation — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 12

Phase 1 — Configure . . . 13

Phase 2 — Pilot . . . 15

Phase 3 — Validate . . . 15

Phase 4 — Expand . . . 15

Testing & Validation — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 16
T1. Connection targets the sovereign cloud . . . 16
T2. Discovery script emits valid single-line JSON . . . 17
T3. JSON rules align with discovered keys (case-sensitive) . . . 17
T4. Objects exist in the GCC High tenant . . . 17
T5. Compliant device grades as Compliant . . . 17
T6. Deliberately non-compliant device grades as Not compliant . . . 17
T7. Noncompliance actions and grace period behave as configured . . . 17
T8. Proxy does not break TLS on the sovereign device endpoints . . . 17
T9. Scope-targeting preview is accurate before any write . . . 18
Troubleshooting — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 18
1. Device shows "not evaluated" / no custom-compliance result (the #1 GCC High issue) . . . 18
2. Sign-in works but scripts/portal "see nothing" — wrong cloud . . . 18
3. Discovery script runs but device never grades Compliant/NonCompliant . . . 19
4. Device grades wrong (false compliant / false non-compliant) . . . 19
5. Can't create or edit the discovery script (Scripts → Add blocked) . . . 19
6. Graph Explorer won't connect to the government tenant . . . 19
7. `Connect-MgGraph -Environment USGov` fails or scopes are missing . . . 20
8. A capability from commercial docs doesn't exist in your tenant . . . 20
9. Users see the wrong / unreachable remediation link . . . 20
10. Conditional Access lockout after enforcement . . . 20
Best Practices — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 21
Sovereign-cloud discipline (GCC High) . . . 21
Discovery script hygiene . . . 21
JSON rules design . . . 22
Naming, scope, and change control . . . 22
Rollout safety . . . 22
Operations . . . 22
Deliverables Summary — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition 23
Documents . . . 23
Scripts (all connect to US Gov Graph with `-Environment USGov`) . . . 23
Companion artifacts you create . . . 24
The order to use them . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

The problem

Intune's built-in device-compliance settings don't cover every control a government organization must enforce — a required security-agent version, a registry hardening value, a TPM/BIOS attribute, a third-party app baseline. **Custom compliance** closes that gap: a PowerShell discovery script plus a JSON rules file let Intune grade any value you can read from a Windows device, and drive Conditional Access from the result.

On a **GCC High** (US sovereign cloud) tenant, doing this correctly is not the commercial procedure with a different login. The admin portals, the device service endpoints, and the Microsoft Graph API are all `*.us` **/ US Gov** URLs, some capabilities lag or differ from commercial, and the failure mode is often **silent** — a device shows "not evaluated" rather than an error. Applying commercial guidance to a government tenant produces exactly this quiet breakage.

The solution

This is the **Government (GCC High) edition** of the Intune Custom Compliance & Configuration Items solution. It re-authors the entire deployment for the US sovereign cloud:

- **Correct sovereign portals and endpoints throughout** — `intune.microsoft.us`, `entra.microsoft.us`, `portal.azure.us`, `login.microsoftonline.us`, `graph.microsoft.us`, `*.manage.microsoft.us` — with an explicit **Commercial vs GCC High** map so nothing is applied to the wrong cloud.
- **Ordered, click-by-click implementation** with GCC-High-specific caveats called out **at the step where they matter** (the Default-scope-tag requirement for scripts; SSL inspection unsupported on `*.manage.microsoft.us`; Graph Explorer unavailable; feature-lag checks).
- **Five PowerShell scripts** that connect to **US Gov Graph** with `-Environment USGov` — discovery, validation, reporting, safe remediation, and scope/target-group building — read-only by default, with remediation and scope changes gated behind `-WhatIf`.
- A **cited GCC High parity reference** and a **testing/troubleshooting** playbook with pass/fail gates.

How it's delivered

The standard, proven model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a gate at the end of each phase. Nothing scales to production until it's proven on a pilot group inside the government tenant — the only reliable way to confirm the mechanics actually grade in GCC High, given that Microsoft documents custom compliance generally rather than GCC-High-specifically.

The outcome

Intune Custom Compliance deployed correctly on GCC High: right portals, right endpoints, parity gaps handled, and validated on a pilot before it touches the fleet. A novice government admin can follow it end-to-end; an experienced one gets the sovereign-cloud gotchas documented instead of discovered in production.

> ***Assurance:*** *every URL and behavior is either cited to current Microsoft Learn documentation (retrieved July 2026) or explicitly flagged* ***"verify for GCC High."*** *Nothing is invented. See* `gcc-high-parity-notes.md` *for the full cited reference.*

Solution Overview — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

What this solution delivers

This solution takes **Intune Custom Compliance & Configuration Items (Scripted)** and re-authors it for a **US sovereign cloud (GCC High)** tenant. Custom compliance lets you extend Intune device-compliance evaluation beyond the built-in settings: you upload a **PowerShell discovery script** plus a **JSON rules file**, Intune runs the script on each targeted Windows device, and the returned values are graded against the JSON. Devices that fail can be marked **not compliant**, which then drives Conditional Access, notifications, and remediation.

On GCC High this is **not** the commercial portal with a different sign-in. The admin consoles, the device service endpoints, and the Microsoft Graph API you script against are all `*.us` **/ US Gov** URLs, and some capabilities lag or differ from commercial. This edition gives you the correct sovereign portals and endpoints, the exact ordered steps, and an explicit map of where commercial guidance would silently mislead you.

You get:

- A **discovery script** authored for Windows devices, packaged and ready to upload.
- A **JSON rules file** pattern (SettingName / Operator / DataType / Operand / MoreInfoUrl / RemediationStrings).
- Five **PowerShell scripts** that connect to **US Gov Graph** (`-Environment USGov`) to discover, validate, report, safely remediate, and build the scope/target group — all read-only by default except where explicitly gated behind `-WhatIf`.
- A **testing and troubleshooting** playbook with GCC-High-specific gates.

The deployment model is the standard one used across every solution in this library: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a gate at the end of each phase.

Named objects (create once, reuse everywhere)

Use these names consistently across the portal, the JSON, and the scripts so the solution is auditable and repeatable. Adjust the prefix to your naming standard, but keep it consistent:

Object	Suggested name	Where it lives
Discovery script	`CUSTCOMPG-Discovery-Win`	Intune → Endpoint security → Device compliance → Scripts
Custom compliance policy	`CUSTCOMPG-Compliance-Baseline`	Intune → Devices → Compliance
JSON rules file	`CUSTCOMPG-Rules.json`	Uploaded into the compliance policy
Pilot group	`CUSTCOMPG-Pilot-Devices`	Entra (`entra.microsoft.us`) → Groups
Production group	`CUSTCOMPG-Prod-Devices`	Entra → Groups
Scope tag	`CUSTCOMPG`	Intune → Tenant administration → Roles → Scope tags

Commercial vs GCC High — what's different

This is the section that keeps a government deployment from failing. If you (or a script, or a copied runbook) use commercial URLs on a GCC High tenant, sign-in and API calls fail or return nothing — often **silently**.

Area	Commercial	GCC High (this edition)	Source
Intune admin center	`https://intune.microsoft.com`	`https://intune.microsoft.us/`	[Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints)
Entra admin center	`https://entra.microsoft.com`	`https://entra.microsoft.us`	[Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints)
Azure portal	`https://portal.azure.com`	`https://portal.azure.us/`	[Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments)
M365 admin	`https://admin.microsoft.com`	`https://portal.office365.us/`	[Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints)

Area	Commercial	GCC High (this edition)	Source
Defender portal	`https://security.microsoft.com`	`https://security.microsoft.us`	Microsoft GCC High portal references (verify for GCC High)
Company Portal	`https://portal.manage.microsoft.com`	`https://portal.manage.microsoft.us/`	[Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints)
Sign-in / auth	`login.microsoftonline.com`	`login.microsoftonline.us`	[Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments)
Microsoft Graph root	`https://graph.microsoft.com`	`https://graph.microsoft.us` (L4/GCC High)	[Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments)
Graph PowerShell connect	`Connect-MgGraph` (Global)	`Connect-MgGraph -Environment USGov`	[Connect-MgGraph](https://learn.microsoft.com/en-us/powershell/module/microsoft.graph.authentication/connect-mggraph)
Device service endpoints	`*.manage.microsoft.com`	`*.manage.microsoft.us`	[Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints)

Area	Commercial	GCC High (this edition)	Source
Graph Explorer (browser API tester)	Supported	**Not supported** for `graph.microsoft.us`	[Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments)

Additional GCC High realities to plan around (detailed with citations in `gcc-high-parity-notes.md`):

- **Feature lag.** Microsoft states that certain services and features available in the global service may not be available in national clouds, and government clouds commonly trail commercial by weeks or months. Before relying on any newer custom-compliance behavior, **verify it is live in your GCC High tenant** rather than assuming commercial documentation applies.
- **No Graph Explorer.** You cannot use the browser-based Graph Explorer against `graph.microsoft.us`; do your API recon with the Microsoft Graph PowerShell SDK using `-Environment USGov` (that is exactly what the five scripts in this solution do).
- **No third-party MDM/MAM integration** in GCC High — plan to do everything natively in Intune.
- **SSL inspection is not supported** on `*.manage.microsoft.us`; if your proxy breaks TLS, discovery-script delivery and compliance check-in will fail.

Who this is for

Government tenants (DoD contractors, agencies, CMMC-scoped organizations) running Microsoft 365 GCC High that need to enforce compliance conditions Intune's built-in settings don't cover — for example, a required security agent version, a registry hardening value, a TPM/BIOS attribute, or a third-party app baseline — and need it done with the **correct sovereign portals and endpoints**, proven on a pilot before scaling.

> ***Sourcing note:*** *URLs and behaviors marked with a Microsoft Learn citation are confirmed against current Microsoft documentation (retrieved July 2026). Items marked *"verify for GCC High"* are drawn from Microsoft government-portal references that Microsoft has not re-confirmed on a first-party Learn page for every capability; confirm them live in your own tenant before production rollout. Nothing in this solution is invented.*

Requirements — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

> *The **category-level** prerequisites (Entra tenant, Intune subscription, admin roles, enrollment, pilot group, supported Windows baseline, network access) are covered once in*

> `global-requirements-intune-solutions.md`. *Confirm those first. The requirements below are **specific to custom compliance on a GCC High tenant** and to this solution's scripts. Each entry: **Description · Why · Verify · Implement-if-missing.***

Every **Verify** step uses the **US Gov (`*.us`)** portals and endpoints. If a link, script, or copied runbook points at a `.com` commercial URL, it is wrong for GCC High — stop and correct it.

R1. GCC High tenant (US sovereign cloud), not Commercial or GCC-Moderate

Description: The tenant is a Microsoft 365 **GCC High** (US Government L4) tenant. **Why:** GCC High uses distinct `*.us` admin portals, distinct device service endpoints (`*.manage.microsoft.us`), and the US Gov Microsoft Graph root (`https://graph.microsoft.us`). Commercial and GCC-Moderate use the worldwide endpoints; steps and scripts are not interchangeable ([Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments)). **Verify:** Sign in at `https://intune.microsoft.us/` and `https://portal.azure.us/`. If those load your tenant (and `https://intune.microsoft.com` does **not**), you are in GCC High. **Implement:** This solution assumes an existing GCC High tenant; provisioning GCC High is a Microsoft licensing/onboarding process outside this solution's scope. Confirm with your Microsoft government account team.

R2. US Government Intune licensing (G3/G5 Gov)

Description: An Intune-bearing US Government plan — typically Microsoft 365 **G3/G5**, or EM+S E3/E5 for Government, or standalone Intune for Government. **Why:** Custom compliance is an Intune device-compliance feature; it requires an Intune-licensed, enrolled device. Intune for GCC/GCC High is available standalone or within the Government EM+S E3/E5 SKUs. **Verify:** `https://portal.office365.us/` → **Billing** → **Licenses**; confirm a Government SKU that includes Intune. In `https://intune.microsoft.us/` → **Tenant administration** → **Tenant status** shows active. **Implement:** Assign a Government Intune SKU to the pilot users/devices before targeting them.

R3. Roles: Intune Administrator (+ default scope tag) in the GCC High tenant

Description: The operator holds **Intune Administrator** in Entra, and — because the discovery-script upload workflow does **not** support scope tags — is assigned the **default** scope tag. **Why:** Creating scripts and compliance policies requires elevated Intune rights. Microsoft explicitly notes that to **create, edit, or view custom compliance discovery scripts you must be assigned the default scope tag**, and the script upload workflow doesn't support scope tags ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script)). **Verify:** `https://entra.microsoft.us` → **Roles and administrators** → confirm **Intune Administrator**. In `https://intune.microsoft.us/` → **Tenant administration** → **Roles** → **Scope (Tags)** → confirm your admin has the **Default** scope tag. **Implement:** Assign **Intune Administrator**; ensure the Default scope tag is present on the admin's role assignment. Reserve Global Administrator for tenant-wide changes only.

R4. Microsoft Graph PowerShell SDK connecting to US Gov

Description: The **Microsoft.Graph** PowerShell SDK installed on the operator workstation, used with `-Environment USGov`. **Why:** All five scripts in this solution talk to `https://graph.microsoft.us`. On GCC High you connect with `Connect-MgGraph -Environment USGov`; the default (Global) connection targets `graph.microsoft.com` and will not see your government tenant ([Connect-MgGraph](https://learn.microsoft.com/en-us/powershell/module/microsoft.graph.authentication/connect-mggraph)). **Graph Explorer is not supported for** `graph.microsoft.us`, so the PowerShell SDK is your API recon tool ([Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments)). **Verify:** In PowerShell run `Get-Module Microsoft.Graph -ListAvailable`. Then test: `Connect-MgGraph -Environment USGov -Scopes "DeviceManagementConfiguration.Read.All"` and confirm `(Get-MgContext).Environment` returns **USGov**. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. Confirm USGov is a valid environment with `Get-MgEnvironment`.

R5. Graph permission scopes for discovery, validation, reporting, remediation

Description: Delegated (or app) Graph scopes for reading and writing device-management configuration and compliance. **Why:** The scripts read compliance policies, scripts, devices, and groups, and (for the remediation/scope scripts, only when you disable `-WhatIf`) write them. **Verify:** After `Connect-MgGraph -Environment USGov`, run `(Get-MgContext).Scopes` and confirm the scopes below are present. **Implement:** Reconnect requesting the scopes:

- **Read-only** (Discovery, Validation, Report scripts): `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Group.Read.All`.
- **Write** (Remediation, Set-ScopeTargeting scripts, only when applying changes): `DeviceManagementConfiguration.ReadWrite.All`, `Group.ReadWrite.All`.

Consent to these in `https://entra.microsoft.us` if an admin-consent prompt appears.

R6. A tested Windows discovery script + JSON rules file

Description: A PowerShell **discovery script** (Windows) and a matching **JSON rules file** defining the compliant conditions. **Why:** Custom compliance requires both: the script returns discovered values as **compressed single-line JSON** (`ConvertTo-Json -Compress`), and the JSON rules file grades them. Each discovery script pairs with **exactly one** compliance policy, and each policy takes **exactly one** discovery script ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script)). **Why the limits matter:** Script ≤ **1 MB**; output ≤ **1 MB**; Windows script must finish in ≤ **10 minutes**. Exceeding these makes the device fail to return data. **Verify:** Run the discovery script locally on a representative Windows device and confirm it emits a single compressed JSON line whose keys exactly match the `SettingName` values in the JSON rules file (case-sensitive). **Implement:** Use this solution's discovery-script pattern and `CUSTCOMPG-Rules.json` template; the `CUSTCOMPG-Validation.ps1` script checks key/type alignment before you upload.

R7. Sovereign network endpoints allowed through filtering (no TLS break)

Description: Outbound access from managed devices to the **US Gov** Intune service endpoints, with **no SSL inspection** on `*.manage.microsoft.us`. **Why:** Discovery scripts are delivered and compliance results returned over `*.manage.microsoft.us`; sign-in uses `login.microsoftonline.us`; device registration uses `enterpriseregistration.microsoftonline.us`. Microsoft states **SSL inspection is not supported on** `*.manage.microsoft.us` — a TLS-breaking proxy will cause silent check-in/compliance failures ([Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints)). **Verify:** From a managed device, confirm reachability of `https://portal.manage.microsoft.us/` and `https://login.microsoftonline.us`, and confirm the proxy is **not** intercepting/re-signing `*.manage.microsoft.us`. **Implement:** Allow `*.manage.microsoft.us`, `login.microsoftonline.us`, `enterpriseregistration.microsoftonline.us`, `graph.microsoft.us`, and `*.events.data.microsoft.com` through proxy/firewall; exempt `*.manage.microsoft.us` from SSL inspection. For EPM/Defender security-management, also allow `*.dm.microsoft.us`.

R8. Windows devices enrolled in the GCC High tenant, in scope

Description: Target Windows 10 21H2+ / Windows 11 devices enrolled in **this** GCC High tenant and members of the pilot group. **Why:** Compliance policies only reach enrolled, targeted devices; custom compliance for Windows requires a PowerShell-capable Windows client. **Verify:** `https://intune.microsoft.us/` → **Devices** → **All devices** → target device appears, is **managed**, and is a member of `CUSTCOMPG-Pilot-Devices`. **Implement:** Enroll via Automatic Enrollment / Entra join (see `global-requirements`), then add to the pilot group with `CUSTCOMPG-Set-ScopeTargeting.ps1` (run with `-WhatIf` first).

Gate to leave Assess: R1–R8 all **Verify** green, the discovery script emits valid compressed JSON matching the rules file, and `Connect-MgGraph -Environment USGov` returns `Environment = USGov`. Only then proceed to **Configure**.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Implementation — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

This is the ordered, click-by-click implementation using the **GCC High (`*.us`)** portals. Every navigation path below is the **US Government** admin center. GCC-High-specific caveats are called out **inline at the step where they bite**, not buried at the end.

Deployment model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a gate after each phase.

> ***Before you start:*** *finish the* ***Assess*** *phase in* `02-requirements.md`*. You should already have: an Intune Administrator sign-in that works at* `https://intune.microsoft.us/`*, the Microsoft Graph PowerShell SDK connecting with* `-Environment USGov`*, and a discovery script that emits valid*

compressed JSON on a test device.

Phase 1 — Configure

Step 1.1 — Create the pilot and production groups (Entra US Gov)

1. Go to `https://entra.microsoft.us` → **Groups** → **All groups** → **New group**.

2. Group type **Security**. Name it `CUSTCOMPG-Pilot-Devices`. Membership type **Assigned** (or Dynamic if you prefer a rule). Create.

3. Repeat to create `CUSTCOMPG-Prod-Devices`.

4. Add 3–10 representative test devices to `CUSTCOMPG-Pilot-Devices`.

GCC High caveat: *this is* `entra.microsoft.us`*,* ***not*** `entra.microsoft.com`*. A bookmark to the commercial portal will sign you into the wrong cloud (or nowhere). Reuse these exact group names in the JSON scoping and in* `CUSTCOMPG-Set-ScopeTargeting.ps1`*.*

Step 1.2 — Create the scope tag (optional but recommended)

1. `https://intune.microsoft.us/` → **Tenant administration** → **Roles** → **Scope (Tags)** → **Create**.

2. Name it `CUSTCOMPG`. Assign later to the compliance policy for RBAC/reporting hygiene.

GCC High caveat (important): *the* ***discovery-script upload workflow does not support scope tags****, and you must be assigned the* ***Default*** *scope tag to create/edit/view discovery scripts ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script)). So the* `CUSTCOMPG` *scope tag applies to the* ***policy****, not the script. Confirm your admin holds the Default scope tag (Requirement R3) or the* ***Scripts*** *→* ***Add*** *blade will not let you proceed.*

Step 1.3 — Finalize the discovery script and JSON rules file

1. On a representative Windows device, run your discovery script (this solution's pattern) and confirm the **last line** is `return $hash | ConvertTo-Json -Compress` — the output must be **one compressed line** ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script)).

2. Open `CUSTCOMPG-Rules.json` and confirm every rule's `SettingName` **exactly** matches (case-sensitive) a key returned by the script, and each rule's `DataType` matches the value's type.

3. Run `CUSTCOMPG-Validation.ps1` to catch key/type mismatches, JSON schema errors, and size/runtime limit risks before upload.

JSON rules file — the shape Intune expects. Each rule object carries: `SettingName`, `Operator` (e.g. `IsEquals`, `GreaterEquals`), `DataType` (e.g. `Boolean`, `String`, `Int64`, `Version`), `Operand` (the compliant value), a `MoreInfoUrl`, and localized `RemediationStrings` (Title + Description shown to the user in Company Portal). Example skeleton:

```
{
  "Rules": [
    {
      "SettingName": "TPMChipPresent",
      "Operator": "IsEquals",
```

```
      "DataType": "Boolean",
      "Operand": true,
      "MoreInfoUrl": "https://aka.ms/compliance",
      "RemediationStrings": [
        {
          "Language": "en_US",
          "Title": "TPM chip must be present and enabled",
          "Description": "This device must have a TPM present. Contact the service desk."
        }
      ]
    }
  ]
}
```

GCC High caveat: *point* `MoreInfoUrl` *at an internal/government-appropriate help page, not a commercial link that GCC High users may not be able to reach. Intune does* ***not*** *validate the discovery script for syntax — test it yourself.*

Step 1.4 — Add the discovery script to Intune

1. `https://intune.microsoft.us/` → **Endpoint security** → **Device compliance** → **Scripts** → **Add** → **Windows**.

2. **Basics:** Name `CUSTCOMPG-Discovery-Win`. Add a clear description.

3. **Settings** → **Detection script:** paste the finalized discovery script. Configure the toggles:

• **Run this script using the logged on credentials** — leave **No** to run as System unless your check needs the user context.

• **Enforce script signature check** — set **Yes** only if you sign the script; otherwise **No**.

• **Run script in 64-bit PowerShell Host** — set **Yes** if your discovery calls 64-bit-only cmdlets/paths.

1. **Review + create.**

GCC High caveat: *navigation is identical to commercial, but you must be in* `intune.microsoft.us`*.* ***Each discovery script binds to exactly one compliance policy*** *— do not try to reuse* `CUSTCOMPG-Discovery-Win` *across multiple policies.*

Step 1.5 — Create the custom compliance policy and attach script + JSON

1. `https://intune.microsoft.us/` → **Devices** → **Compliance** → **Create policy** → Platform **Windows 10 and later** → **Create**.

2. **Basics:** Name `CUSTCOMPG-Compliance-Baseline`.

3. **Compliance settings** → **Custom Compliance** → **Custom compliance = Require.**

• **Select your discovery script:** choose `CUSTCOMPG-Discovery-Win`.

• **Upload the JSON file:** upload `CUSTCOMPG-Rules.json`.

1. **Actions for noncompliance:** start conservative — **Mark device noncompliant** after a grace period (e.g., 1 day), plus an email notification. Do **not** wire this into Conditional Access yet.

2. **Scope tags:** add `CUSTCOMPG`.

3. **Assignments:** assign to `CUSTCOMPG-Pilot-Devices` only. **Create.**

GCC High caveat: *because Graph Explorer is unavailable in GCC High, use* `CUSTCOMPG-Discovery.ps1` *(with* `-Environment USGov`*) to confirm the policy and script objects*

exist as expected instead of poking the API in a browser.

Gate to leave Configure: the script `CUSTCOMPG-Discovery-Win` and policy `CUSTCOMPG-Compliance-Baseline` exist, the JSON uploaded without error, and assignment targets **only** the pilot group.

Phase 2 — Pilot

Step 2.1 — Sync and force evaluation

1. On a pilot device: **Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**, or from `https://intune.microsoft.us/` → **Devices** → **All devices** → **[device]** → **Sync**.
2. Allow time for the discovery script to run and results to return over `*.manage.microsoft.us`.

> ***GCC High caveat:*** *if the device's proxy* ***breaks TLS*** *on `*.manage.microsoft.us`, results never return and the device shows no custom-compliance status (not a failure — a silence). This is the #1 GCC High gotcha. Confirm R7 (SSL inspection exempted).*

Step 2.2 — Confirm results with the reporting script

1. Run `CUSTCOMPG-Report.ps1` (connects `-Environment USGov`) to pull per-device compliance state for `CUSTCOMPG-Compliance-Baseline` across the pilot group.
2. Cross-check in `https://intune.microsoft.us/` → **Devices** → **Compliance** → `CUSTCOMPG-Compliance-Baseline` → **Device status**.

Gate to leave Pilot: at least one pilot device reports a **Compliant** and one deliberately-failing device reports **Not compliant** for the expected setting — proving the script *and* the JSON grading both work end-to-end in GCC High.

Phase 3 — Validate

Run the full validation playbook in `05-testing.md`. At minimum:

- Discovery output matches JSON keys (case-sensitive) on real devices — `CUSTCOMPG-Validation.ps1`.
- A known-bad device is correctly flagged **Not compliant** with the right remediation string in Company Portal (`portal.manage.microsoft.us`).
- Grace period and noncompliance actions behave as configured.
- No devices are stuck in "no result" (the TLS-inspection symptom).

Gate to leave Validate: every test in `05-testing.md` passes on the pilot group; false-positive/false-negative rate is zero on the deliberate test cases.

Phase 4 — Expand

Step 4.1 — Widen scope in controlled waves

1. Use `CUSTCOMPG-Set-ScopeTargeting.ps1` (default `-WhatIf` — it only *previews* membership changes) to stage devices into `CUSTCOMPG-Prod-Devices` in waves.

2. Re-run with `-WhatIf:$false` **only after** reviewing the preview.

3. In `https://intune.microsoft.us/` → **Devices** → **Compliance** → `CUSTCOMPG-Compliance-Baseline` → **Assignments**, add `CUSTCOMPG-Prod-Devices`.

Step 4.2 — Only now, wire into Conditional Access (optional)

1. After a clean validation window, in `https://entra.microsoft.us` → **Protection** → **Conditional Access**, require **compliant device** for the target apps — expanding gradually.

> ***GCC High caveat:*** *verify the specific Conditional Access + compliance behavior is live in **your** GCC High tenant before enforcing — government clouds can lag commercial (see `gcc-high-parity-notes.md`). Test with a pilot account first; a misconfigured CA rule can lock users out.*

Step 4.3 — Report and hand off

1. Schedule `CUSTCOMPG-Report.ps1` for periodic compliance snapshots.

2. Keep the discovery script, JSON, and group names under change control so the solution stays auditable.

Gate to leave Expand: production group fully assigned, reporting scheduled, Conditional Access (if used) enforced only after a clean validation window, and rollback steps in `06-troubleshooting.md` are known to the operators.

Testing & Validation — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

Run this playbook against the **pilot** group before expanding. Every check uses the **GCC High (`*.us`)** portals and **US Gov Graph** (`-Environment USGov`). Each test has an explicit **pass/fail gate**. Do not proceed to Expand until all gates pass.

> ***Golden rule for GCC High:*** *a "no result" is **not** the same as "compliant." On a sovereign tenant the most common cause of a device showing no custom-compliance status is a **TLS-breaking proxy on** `*.manage.microsoft.us` (SSL inspection is unsupported there). Treat missing results as a failure to investigate, not a pass.*

T1. Connection targets the sovereign cloud

Do: `Connect-MgGraph -Environment USGov -Scopes "DeviceManagementConfiguration.Read.All"`, then `(Get-MgContext).Environment`. **Pass:** returns `USGov`. **Fail:** returns `Global` (you connected to commercial `graph.microsoft.com`) — reconnect with `-Environment USGov`. **Reference:** [Connect-MgGraph](https://learn.microsoft.com/en-us/powershell/module/microsoft.graph.authentication/connect-mggraph).

T2. Discovery script emits valid single-line JSON

Do: Run the discovery script locally on a representative Windows device. **Pass:** Output is **one compressed JSON line** (from `ConvertTo-Json -Compress`), e.g. `{"BiosVersion":"1.24","TPMChipPresent":true}`; script finishes in ≤ **10 minutes**; script file ≤ **1 MB**; output ≤ **1 MB**. **Fail:** Multi-line output, missing keys, or over any limit — Intune won't grade it correctly. **Reference:** [Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script).

T3. JSON rules align with discovered keys (case-sensitive)

Do: Run `CUSTCOMPG-Validation.ps1` against the discovery output and `CUSTCOMPG-Rules.json`. **Pass:** Every rule `SettingName` matches a returned key **exactly** (case-sensitive), and each `DataType` matches the value's type (`Boolean`/`String`/`Int64`/`Version`). **Fail:** Any mismatch — fix the JSON or the script so keys and types line up. A case difference alone will cause a silent grading failure.

T4. Objects exist in the GCC High tenant

Do: Run `CUSTCOMPG-Discovery.ps1` (`-Environment USGov`). **Pass:** It lists the script `CUSTCOMPG-Discovery-Win`, the policy `CUSTCOMPG-Compliance-Baseline` (with a custom-compliance setting), and the pilot group `CUSTCOMPG-Pilot-Devices`. **Fail:** Missing objects — revisit Configure (Phase 1). (Remember: Graph Explorer can't be used against `graph.microsoft.us`, so this script *is* your API recon.)

T5. Compliant device grades as Compliant

Do: On a known-good pilot device, **Sync**, wait, then check `https://intune.microsoft.us/` → **Devices** → **Compliance** → `CUSTCOMPG-Compliance-Baseline` → **Device status**, and cross-check with `CUSTCOMPG-Report.ps1`. **Pass:** Device shows **Compliant** for the custom setting. **Fail:** Shows "not evaluated" / no result → suspect TLS inspection on `*.manage.microsoft.us` (see T8) or an unenrolled/unlicensed device.

T6. Deliberately non-compliant device grades as Not compliant

Do: On a second pilot device, force the checked condition to fail (e.g., set the value below the required baseline). **Sync**, wait, re-check. **Pass:** Device shows **Not compliant** for the expected `SettingName`, and the **RemediationStrings** Title/Description appear in Company Portal at `https://portal.manage.microsoft.us/`. **Fail:** Still compliant → operator/DataType mismatch in the JSON, or the script isn't returning that key.

T7. Noncompliance actions and grace period behave as configured

Do: Confirm the grace period and the "Mark device noncompliant" / notification action fire on the failing device per your policy settings. **Pass:** Action and timing match what you configured in Step 1.5. **Fail:** Re-check **Actions for noncompliance** on the policy.

T8. Proxy does not break TLS on the sovereign device endpoints

Do: From a pilot device, reach `https://portal.manage.microsoft.us/` and `https://login.microsoftonline.us`; confirm the proxy is **not** re-signing certificates for `*.manage.microsoft.us`. **Pass:** Endpoints reachable; certificate chain for `*.manage.microsoft.us` is Microsoft's (not your proxy's). **Fail:** Proxy is intercepting `*.manage.microsoft.us` → exempt it from SSL inspection (Requirement R7). This is the leading GCC High cause of missing compliance results. **Reference:** [Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints).

T9. Scope-targeting preview is accurate before any write

Do: Run `CUSTCOMPG-Set-ScopeTargeting.ps1` in its default `-WhatIf` mode. **Pass:** The preview lists exactly the devices you intend to add/remove from the target group; **no changes are made**. **Fail:** Preview shows unexpected devices → refine the input before ever running with `-WhatIf:$false`.

Overall validation gate: T1–T9 all pass on the pilot group, with at least one true Compliant and one true Not-compliant result and **zero** "no result" devices. Only then move to **Expand**.

Troubleshooting — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

Failure modes and fixes, ordered by how often they bite on a **GCC High** tenant. Almost every GCC High problem is one of three things: **you're in the wrong cloud**, **TLS is being broken on the device endpoints**, or **the script output doesn't match the JSON**. Each item below is symptom → cause → fix.

1. Device shows "not evaluated" / no custom-compliance result (the #1 GCC High issue)

Symptom: In `https://intune.microsoft.us/` → policy → **Device status**, the device is `unknown` / `not applicable`; `CUSTCOMPG-Report.ps1` flags it as NotEvaluated. No error — just silence. **Most likely cause:** a proxy is performing **SSL inspection on** `*.manage.microsoft.us`. Microsoft does not support TLS interception on these endpoints, so discovery-script delivery and result return fail silently ([Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints)). **Fix:** Exempt `*.manage.microsoft.us` (and `*.dm.microsoft.us` for EPM/Defender) from SSL inspection. Confirm the certificate the device sees for `*.manage.microsoft.us` is Microsoft's, not your proxy's. Re-sync and re-check. **Other causes to rule out:** device not licensed for Intune (R2), not enrolled/targeted (R8), or the policy not yet assigned to the device's group.

2. Sign-in works but scripts/portal "see nothing" — wrong cloud

Symptom: The portal or a script returns an empty tenant, or `Connect-MgGraph` succeeds but returns no GCC High data. **Cause:** You're pointed at the **Commercial** cloud (`.com` / Global) instead of GCC High. **Fix:**

• Portals: use `intune.microsoft.us`, `entra.microsoft.us`, `portal.azure.us`, `portal.office365.us` — never the `.com` equivalents.

• Scripts: reconnect with `Connect-MgGraph -Environment USGov` and confirm `(Get-MgContext).Environment` returns `USGov` ([Connect-MgGraph](https://learn.microsoft.com/en-us/powershell/module/microsoft.graph.authentication/connect-mggraph)). All five scripts in this solution check this automatically and stop if it isn't `USGov`.

3. Discovery script runs but device never grades Compliant/NonCompliant

Symptom: Script works locally, but the policy never reflects a result. **Causes and fixes:**

• **Output not single-line.** The last line must be `return $hash | ConvertTo-Json -Compress`. Multi-line output isn't graded ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script)).

• **Over a limit.** Script > 1 MB, output > 1 MB, or runtime > 10 minutes (Windows). Trim what you discover.

• **Wrong host bitness.** A 64-bit-only cmdlet/path under the default 32-bit host returns nothing — set **Run script in 64-bit PowerShell Host = Yes**.

• **User-context mismatch.** If the check needs the logged-on user but runs as System (or vice-versa), it returns empty — align the "Run using logged-on credentials" toggle.

4. Device grades wrong (false compliant / false non-compliant)

Symptom: A device that should fail passes, or vice-versa. **Cause:** Mismatch between discovery output and the JSON rules. **Fix:** Run `CUSTCOMPG-Validation.ps1`. Check:

• **Case-sensitive key match** — `SettingName` must match the returned key **exactly**. `TPMChipPresent` ≠ `TpmChipPresent`.

• **Correct `DataType`/`Operator`** — use `Version` (not `String`) for version comparisons so `GreaterEquals` works; `Boolean` for true/false; `Int64` for counts.

• `Operand` is the intended compliant value.

5. Can't create or edit the discovery script (Scripts → Add blocked)

Symptom: The **Endpoint security** → **Device compliance** → **Scripts** → **Add** workflow won't proceed, or you can't see existing scripts. **Cause:** The discovery-script workflow **does not support scope tags**, and you must hold the **Default** scope tag to create/edit/view these scripts ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script)). **Fix:** In `intune.microsoft.us` → **Tenant administration** → **Roles**, ensure your admin's role assignment includes the **Default** scope tag (Requirement R3). Apply the `CUSTCOMPG` scope tag to the **policy**, not the script.

6. Graph Explorer won't connect to the government tenant

Symptom: Browser-based Graph Explorer returns nothing / errors for `graph.microsoft.us`. **Cause:** **Graph Explorer is not supported** for the US Gov Graph endpoint ([Graph national

clouds](https://learn.microsoft.com/en-us/graph/deployments)). **Fix:** Use the Microsoft Graph PowerShell SDK with `-Environment USGov` for all API recon — `CUSTCOMPG-Discovery.ps1` does exactly this.

7. `Connect-MgGraph -Environment USGov` fails or scopes are missing

Symptom: Connection error, or `(Get-MgContext).Scopes` lacks a needed permission. **Fixes:**

- **SDK missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`; confirm `Get-MgEnvironment` lists `USGov`.
- **Missing scopes:** reconnect requesting them — read-only work needs `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Group.Read.All`; writes need `DeviceManagementConfiguration.ReadWrite.All`, `Group.ReadWrite.All`.
- **Admin consent:** approve the consent prompt in `entra.microsoft.us`.

8. A capability from commercial docs doesn't exist in your tenant

Symptom: A custom-compliance, Linux, or Conditional Access option in commercial documentation isn't present in GCC High. **Cause:** National clouds may lack some global-service features, and government clouds can lag commercial ([Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments)). **Fix:** Treat commercial docs as **"verify for GCC High"** (see `gcc-high-parity-notes.md`). Confirm the capability is live in your tenant on the pilot before depending on it; if absent, plan around it or check availability with your Microsoft government account team.

9. Users see the wrong / unreachable remediation link

Symptom: In Company Portal (`portal.manage.microsoft.us`), the "more info" link doesn't resolve for government users. **Cause:** `MoreInfoUrl` in the JSON points at a commercial or externally-hosted page the government environment can't reach. **Fix:** Set `MoreInfoUrl` (and any operator help links) to an internal/US-Gov-appropriate resource, and write actionable `RemediationStrings` that route users to the government service desk.

10. Conditional Access lockout after enforcement

Symptom: After requiring "compliant device" in Conditional Access, users are blocked. **Cause:** CA enforced before validation was clean, or before compliance results were reliably returning (see #1). **Fix:** Roll back the CA rule in `entra.microsoft.us`. Confirm devices are genuinely grading Compliant (clean `CUSTCOMPG-Report.ps1`), then re-enable CA gradually with a pilot account and a break-glass exclusion. Only wire CA in after a clean validation window (Implementation Phase 4).

When in doubt, in this order: confirm you're in **USGov** (portals and `Get-MgContext`), confirm `*.manage.microsoft.us` **isn't SSL-inspected**, then run `CUSTCOMPG-Validation.ps1` to confirm script output matches the JSON. Those three cover the large majority of GCC High custom-compliance failures.

Best Practices — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

Practices that keep custom compliance reliable, auditable, and safe **on a GCC High tenant**. The sovereign-cloud items are the ones commercial runbooks miss.

Sovereign-cloud discipline (GCC High)

• **Bookmark the `*.us` portals and delete the `.com` ones.** `intune.microsoft.us`, `entra.microsoft.us`, `portal.azure.us`, `portal.office365.us`, `security.microsoft.us`. A single commercial bookmark is how a government admin ends up making changes in the wrong (or no) cloud. ([Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints))

• **Every script call uses `-Environment USGov`.** Bake it into the scripts (this solution does) and verify `(Get-MgContext).Environment` is `USGov` before trusting any output. The default connection silently targets commercial and returns nothing for your tenant. ([Connect-MgGraph](https://learn.microsoft.com/en-us/powershell/module/microsoft.graph.authentication/connect-mggraph))

• **Do API recon with PowerShell, not Graph Explorer.** Graph Explorer isn't supported for `graph.microsoft.us`. ([Graph national clouds](https://learn.microsoft.com/en-us/graph/deployments))

• **Never SSL-inspect `*.manage.microsoft.us`.** Microsoft doesn't support it; breaking TLS there causes silent compliance failures. Exempt these hosts in the proxy and document the exemption. ([Intune US Gov endpoints](https://learn.microsoft.com/en-us/intune/intune-service/fundamentals/intune-us-government-endpoints))

• **Verify feature parity before you rely on anything new.** Government clouds can trail commercial. Confirm the specific custom-compliance/Conditional-Access behavior is live in *your* GCC High tenant rather than assuming commercial docs apply (see `gcc-high-parity-notes.md`).

• **Keep help links government-appropriate.** `MoreInfoUrl` in the JSON and any operator docs should point to internal/US Gov resources users can actually reach.

Discovery script hygiene

• **Return compressed single-line JSON** — the last line must be `ConvertTo-Json -Compress`. Multi-line output fails grading. ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script))

• **Stay inside the limits:** script ≤ 1 MB, output ≤ 1 MB, Windows runtime ≤ 10 minutes. Discover only what you grade — don't dump the whole machine.

• **Run as System unless you need user context.** Only enable "run using logged-on credentials" when the setting is genuinely per-user.

• **Choose the right host bitness.** Enable the 64-bit host when your cmdlets/registry paths are 64-bit-only, to avoid WOW6432 redirection surprises.

• **Make discovery read-only.** A discovery script should observe, never change state. Keep enforcement/remediation in a separate, explicitly-gated path.

• **Intune won't lint your script.** Test on representative hardware/OS builds before upload — Intune validates neither syntax nor logic.

JSON rules design

• **Match keys exactly (case-sensitive)** between script output and `SettingName`. A case difference is a silent failure.

• **Pick the correct** `DataType` **and** `Operator`**.** `Version` for version strings (so `GreaterEquals` compares correctly), `Boolean` for true/false, `Int64` for counts. A `String` compare on a version is a classic false pass.

• **Write remediation strings for the end user.** The `RemediationStrings` Title/Description are what the person sees in Company Portal — make them actionable and route them to the government service desk, not a commercial URL.

• **One script, one policy.** Each discovery script binds to exactly one compliance policy and vice-versa — name and scope accordingly.

Naming, scope, and change control

• **Reuse the** `CUSTCOMPG-*` **names** consistently across portal objects, JSON, and scripts so the solution is auditable.

• **Apply the** `CUSTCOMPG` **scope tag to the policy** (the *script* upload workflow doesn't support scope tags; the admin needs the **Default** scope tag to manage scripts). ([Create discovery scripts](https://learn.microsoft.com/en-us/intune/intune-service/protect/compliance-custom-script))

• **Version the script + JSON in source control.** Treat them as code; review changes before republishing.

Rollout safety

• **Follow Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with the gates. Never skip Pilot on a government tenant.

• **Start noncompliance actions soft.** Mark-noncompliant + notify with a grace period first; add Conditional Access only after a clean validation window — and test CA with a pilot account so a bad rule can't lock out the fleet.

• **Default to** `-WhatIf`**.** The remediation and scope-targeting scripts preview by default; only disable `-WhatIf` after reviewing the preview.

• **Expand in waves,** not all at once, watching the report between waves.

Operations

• **Schedule** `CUSTCOMPG-Report.ps1` for periodic snapshots; investigate any rise in "no result" devices (usually the TLS-inspection symptom).

• **Least privilege.** Read-only scopes for day-to-day reporting; request write scopes only when applying a change.

• **Document the proxy exemptions and endpoint allowlist** so the next operator doesn't re-break TLS on `*.manage.microsoft.us`.

Deliverables Summary — Intune Custom Compliance & Configuration Items (Scripted), Government (GCC High) Edition

Everything included in this solution, and how the pieces fit together for a **GCC High** deployment.

Documents

Document	What it gives you
`executive-summary.md`	The problem, the sovereign-cloud solution, and the outcome — for decision-makers.
`01-solution-overview.md`	What custom compliance is on GCC High, named objects to reuse, and the **Commercial vs GCC High — what's different** map.
`02-requirements.md`	GCC-High-specific prerequisites (R1–R8) with Verify / Implement steps. Category-level basics live in `global-requirements-intune-solutions.md`.
`03-implementation.md`	Ordered, click-by-click deployment in the `*.us` portals, with GCC-High caveats inline. Assess → Configure → Pilot → Validate → Expand.
`05-testing.md`	Nine pass/fail validation tests (T1–T9) run against the pilot group.
`06-troubleshooting.md`	GCC-High-specific failure modes and fixes (silent "no result," wrong cloud, TLS inspection, scope-tag block).
`07-best-practices.md`	Sovereign-cloud discipline, discovery-script hygiene, JSON design, rollout safety, operations.
`gcc-high-parity-notes.md`	Cited Commercial-vs-GCC-High reference; every row cited or flagged "verify for GCC High."
`08-deliverables-summary.md`	This document.

Scripts (all connect to US Gov Graph with `-Environment USGov`)

Script	Mode	Purpose
`CUSTCOMPG-Discovery.ps1`	**Read-only**	Inventory discovery scripts, custom-compliance policies, and target groups in the GCC High tenant. Your API-recon tool (Graph Explorer is unavailable for `graph.microsoft.us`).
`CUSTCOMPG-Validation.ps1`	**Read-only** (local)	Pre-flight the discovery script + JSON: single-line compressed output, key/DataType alignment (case-sensitive), and Intune size/runtime limits — before upload.
`CUSTCOMPG-Report.ps1`	**Read-only**	Per-device compliance status for the policy, highlighting "not evaluated" devices (the GCC High TLS-inspection symptom); optional CSV.
`CUSTCOMPG-Remediation.ps1`	`-WhatIf` **by default**	Reconcile the tenant to your source-controlled discovery script; confirm policy linkage. Previews unless run with `-WhatIf:$false`.
`CUSTCOMPG-Set-ScopeTargeting.ps1`	`-WhatIf` **by default**	Stage devices into the pilot/production target groups in controlled waves. Previews membership changes unless run with `-WhatIf:$false`.

Safety model: the three read-only scripts never change tenant state. The two change-making scripts (`Remediation`, `Set-ScopeTargeting`) **preview by default** and only write when you explicitly pass `-WhatIf:$false`. Every script verifies `(Get-MgContext).Environment = USGov` before trusting or changing anything.

Companion artifacts you create

Artifact	Where	Notes
`CUSTCOMPG-Rules.json`	Uploaded into the compliance policy	The JSON rules file (SettingName / Operator / DataType / Operand / MoreInfoUrl / RemediationStrings). Validate with `CUSTCOMPG-Validation.ps1`.
Discovery script (`CUSTCOMPG-Discovery-Win`)	Intune → Endpoint security → Device compliance → Scripts	Windows PowerShell; ends in `ConvertTo-Json -Compress`.

Artifact	Where	Notes
`CUSTCOMPG-Pilot-Devices` / `CUSTCOMPG-Prod-Devices`	Entra (`entra.microsoft.us`) → Groups	Assignment targets for the policy.

The order to use them

1. **Assess** — `02-requirements.md`; confirm `Connect-MgGraph -Environment USGov` returns `USGov`; validate the discovery script + JSON with `CUSTCOMPG-Validation.ps1`.
2. **Configure** — `03-implementation.md` Phase 1; use `CUSTCOMPG-Discovery.ps1` to confirm objects; optionally `CUSTCOMPG-Remediation.ps1 -WhatIf` to preview reconciliation.
3. **Pilot** — assign to `CUSTCOMPG-Pilot-Devices`; check results with `CUSTCOMPG-Report.ps1`.
4. **Validate** — run all of `05-testing.md` (T1–T9).
5. **Expand** — stage waves with `CUSTCOMPG-Set-ScopeTargeting.ps1` (preview, then apply); add `CUSTCOMPG-Prod-Devices` to the policy; wire Conditional Access last.

Every URL and behavior in these deliverables is cited to current Microsoft Learn documentation or flagged **"verify for GCC High."** See `gcc-high-parity-notes.md`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CUSTCOMPG-Discovery.ps1

Purpose: Read-only recon for the Government (GCC High) edition of Intune Custom Compliance & Configuration Items. Lists device-compliance discovery scripts, custom-compliance policies, and the pilot/production target groups so an admin can confirm what exists before configuring or expanding. Because Graph Explorer is not supported for graph.microsoft.us, this script is the API-recon tool for GCC High. Replace the placeholder names with your own, e.g. 'CUSTCOMPG-Discovery-Win'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Discovery.ps1 (Graph scopes: DeviceManagementConfiguration.Read.All, Group.Read.All; -Environment USGov) EXPECTED RESULTS A printed inventory of discovery scripts, custom-compliance policies, and matching groups in the GCC High tenant. Pass = objects you expect are listed; Fail = missing objects or a connection whose environment is not USGov. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Discovery.ps1 (Graph scopes: DeviceManagementConfiguration.Read.All, Group.Read.All; -Environment USGov) EXPECTED RESULTS A printed inventory of discovery scripts, custom-compliance policies, and matching groups in the GCC High tenant. Pass = objects you expect are listed; Fail = missing objects or a connection whose environment is not USGov. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

CUSTCOMPG-Remediation.ps1

Purpose: Change-making helper for the Government (GCC High) edition of Intune Custom Compliance, gated behind -WhatIf so it PREVIEWS by default and changes nothing until you opt in. It ensures the named discovery script exists (creating/updating it from a local .ps1), and ensures the named custom-compliance policy references that script and the JSON rules file. Use it to reconcile the tenant to your source-controlled script + rules. Point -ScriptPath at your discovery script (e.g. 'CUSTCOMPG-Discovery-Win.ps1'), -RulesPath at your rules file (e.g. 'CUSTCOMPG-Rules.json'). HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Remediation.ps1 -ScriptPath .\CUSTCOMPG-Discovery-Win.ps1 -RulesPath .\CUSTCOMPG-Rules.json Apply (explicit): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Remediation.ps1 -ScriptPath .\CUSTCOMPG-Discovery-Win.ps1 -RulesPath .\CUSTCOMPG-Rules.json -WhatIf:$false (Graph scopes: DeviceManagementConfiguration.ReadWrite.All; -Environment USGov) EXPECTED RESULTS In preview: a list of the create/update actions that WOULD run, and nothing changes. When applied: the discovery script and policy are reconciled and confirmed. Always run CUSTCOMPG-Validation.ps1 first and pilot before applying broadly. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Remediation.ps1 -ScriptPath .\CUSTCOMPG-Discovery-Win.ps1 -RulesPath .\CUSTCOMPG-Rules.json Apply (explicit): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Remediation.ps1 -ScriptPath .\CUSTCOMPG-Discovery-Win.ps1 -RulesPath .\CUSTCOMPG-Rules.json -WhatIf:$false (Graph scopes:

DeviceManagementConfiguration.ReadWrite.All; -Environment USGov) EXPECTED RESULTS In preview: a list of the create/update actions that WOULD run, and nothing changes. When applied: the discovery script and policy are reconciled and confirmed. Always run CUSTCOMPG-Validation.ps1 first and pilot before applying broadly. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

CUSTCOMPG-Report.ps1

Purpose: Read-only reporting for the Government (GCC High) edition of Intune Custom Compliance. Pulls the device compliance state for the named custom-compliance policy and writes a per-device summary (Compliant / NonCompliant / not evaluated) to the console and an optional CSV. 'Not evaluated' rows are highlighted because on GCC High they usually mean results never returned (commonly a TLS-inspecting proxy on manage.microsoft.us). Point -PolicyName at your policy, e.g. 'CUSTCOMPG-Compliance-Baseline'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Report.ps1 -PolicyName "CUSTCOMPG-Compliance-Baseline" -CsvPath .\custcompg-report.csv (Graph scopes: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All; -Environment USGov) EXPECTED RESULTS A table of devices with their compliance state for the policy, plus counts. Pass = the expected devices report Compliant/NonCompliant; investigate any 'not evaluated' rows. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Report.ps1 -PolicyName "CUSTCOMPG-Compliance-Baseline" -CsvPath .\custcompg-report.csv (Graph scopes: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All; -Environment USGov) EXPECTED RESULTS A table of devices with their compliance state for the policy, plus counts. Pass = the expected devices report Compliant/NonCompliant; investigate any 'not evaluated' rows. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

CUSTCOMPG-Set-ScopeTargeting.ps1

Purpose: Controlled scope/target-group builder for the Government (GCC High) edition of Intune Custom Compliance, gated behind -WhatIf so it PREVIEWS membership changes and changes nothing until you opt in. Adds the devices you supply (by device name or object id) to the named Entra security group used for compliance-policy assignment, so you can expand in controlled waves. Point -GroupName at your group (e.g. 'CUSTCOMPG-Prod-Devices') and supply devices via -DeviceNames or -DeviceObjectIds. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Set-ScopeTargeting.ps1 -GroupName "CUSTCOMPG-Prod-Devices" -DeviceNames "PC-001","PC-002" Apply (explicit): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Set-ScopeTargeting.ps1 -GroupName "CUSTCOMPG-Prod-Devices" -DeviceNames "PC-001","PC-002" -WhatIf:$false (Graph scopes: Group.ReadWrite.All, Device.Read.All; -Environment USGov) EXPECTED RESULTS In preview: the exact add operations that WOULD run, and nothing changes. When applied: the listed devices are added to the target group (existing members are skipped). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Set-ScopeTargeting.ps1 -GroupName "CUSTCOMPG-Prod-Devices" -DeviceNames "PC-001","PC-002" Apply (explicit): powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Set-ScopeTargeting.ps1 -GroupName "CUSTCOMPG-Prod-Devices" -DeviceNames "PC-001","PC-002" -WhatIf:$false (Graph scopes: Group.ReadWrite.All, Device.Read.All; -Environment

USGov) EXPECTED RESULTS In preview: the exact add operations that WOULD run, and nothing changes. When applied: the listed devices are added to the target group (existing members are skipped). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

CUSTCOMPG-Validation.ps1

Purpose: Read-only pre-flight check for the Government (GCC High) edition of Intune Custom Compliance. Runs (or reads the output of) the Windows discovery script and confirms it emits a single compressed JSON line whose keys exactly match the SettingName values in the JSON rules file (case-sensitive), that DataTypes line up, and that Intune size and runtime limits are respected. Catches the silent grading failures before you upload. Point -RulesPath at your rules file, e.g. 'CUSTCOMPG-Rules.json', and -ScriptPath at your discovery script, e.g. 'CUSTCOMPG-Discovery-Win.ps1'. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Validation.ps1 -ScriptPath .\CUSTCOMPG-Discovery-Win.ps1 -RulesPath .\CUSTCOMPG-Rules.json (No Graph connection required — local validation. When checking the tenant, connect with -Environment USGov.) EXPECTED RESULTS A PASS/FAIL report per rule. Pass = every SettingName maps to a returned key with a matching DataType and all limits are within bounds; Fail = a mismatch, missing key, multi-line output, or a size/runtime breach that would break grading in GCC High. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>

How to run: powershell -ExecutionPolicy Bypass -File .\CUSTCOMPG-Validation.ps1 -ScriptPath .\CUSTCOMPG-Discovery-Win.ps1 -RulesPath .\CUSTCOMPG-Rules.json (No Graph connection required — local validation. When checking the tenant, connect with -Environment USGov.) EXPECTED RESULTS A PASS/FAIL report per rule. Pass = every SettingName maps to a returned key with a matching DataType and all limits are within bounds; Fail = a mismatch, missing key, multi-line output, or a size/runtime breach that would break grading in GCC High. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph US Gov endpoints. #>